

Airbnb Data Science Project

Luke O'Connell and Yvonne Quach

May 14, 2025

Contents

Introduction	2
Dataset Overview	2
Libraries and Setup	2
Price Optimization	3
Data Preparation	3
Model Building	10
Model Evaluation	12
Price Optimization - Linear Regression Model	12
Model Evaluation	13
Revenue Forecasting	13
Data Preparation	14
Data Engineering	16
Forecasting Models	16
Model Evaluation	18
Market Segmentation	19
General Segmentation	19
Amenities Segmentation	25
Summary of cluster profiles	27
View the summary table of cluster profiles	27
Silhouette score	27
PCA on the scaled amenities clustering data	27

Scree plot on the scaled amenities clustering data	27
ggplot on the scaled amenities clustering data	27
Location Segmentation	28

Introduction

This project applies machine learning techniques to Airbnb listings in Los Angeles County to address three core business objectives, in the hopes of providing hosts data-driven information on how best to market, price, and respond to guest demand:

1. **Price Optimization** – Determine optimal nightly prices for listings based on factors such as room type, location, amenities, and review scores.
2. **Revenue Forecasting** – Predict revenue using booking history of similar rentals, allowing hosts to anticipate seasonal demand.
3. **Market Segmentation** – Cluster listings into distinct market segments to inform business strategies, marketing efforts, and pricing tiers.

The insights aim to support both hosts and platforms in making more data-informed decisions that enhance profitability and customer satisfaction.

Dataset Overview

Libraries and Setup

```
# Always print this out before your assignment
sessionInfo()
## R version 4.4.2 (2024-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sequoia 15.4.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/Los_Angeles
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods
## [7] base
##
## other attached packages:
## [1] knitr_1.49
##
```

```
## loaded via a namespace (and not attached):
## [1] compiler_4.4.2      fastmap_1.2.0      cli_3.6.4
## [4] formatR_1.14       tools_4.4.2       htmltools_0.5.8.1
## [7] rstudioapi_0.17.1  yaml_2.3.10       rmarkdown_2.29
## [10] xfun_0.50          digest_0.6.37     rlang_1.1.5
## [13] evaluate_1.0.3
getwd()
## [1] "/Users/leo/Desktop"
```

```
# Load required libraries
library(dplyr)
library(tidyr)
library(tidyverse)
library(randomForest)
library(caret)
library(stringr)
library(forcats)
library(rsample)
library(glmnet)
library(glmnetUtils)
library(tidyverse)
library(cluster)
library(factoextra)
library(stringr)
library(ggfortify)

# Load the dataset, found here:

LA_listings <- read_csv("/Users/leo/Desktop/BUS659/project/LA_listings.csv",
  show_col_types = FALSE)

# https://insideairbnb.com/get-the-data/ -> 'Los Angeles, California,
# United States' header -> file name: 'listings.csv.gz'. Based on
# March 1, 2025 listings.
```

Price Optimization

Start: Price optimization... Predict optimal nightly rates based on listing's characteristics, like room type, amenities, reviews, and location.

Since there are 3 objectives, all of which will require testing, training, and likely creation of similar data sets, all data groups created for price optimization will begin with "opt_".

Data Preparation

```
#Select relevant columns and clean them for modeling
opt_LA_price <- LA_listings %>%
```

```

select(price, room_type, neighbourhood_cleansed, number_of_reviews, review_scores_rating, minimum_nigh
mutate(

  # Convert price string to numeric
  price = as.numeric(gsub("$,", "", price)),

  # Convert categorical variables to factors
  room_type = as.factor(room_type),
  neighbourhood_cleansed = as.factor(neighbourhood_cleansed),

  #Convert bedroom/bathroom/bed counts to numeric
  bedrooms = as.numeric(bedrooms),
  bathrooms = as.numeric(bathrooms),
  beds = as.numeric(beds),

  # Create a new numeric feature: count of amenities
  amenities_count = str_count(amenities, ",") +1,

  #Price per bedroom (if bedroom is not 0), indicates price based on listing size
  price_per_bedroom = ifelse(bedrooms > 0, price / bedrooms, NA),

  #Price per bed, indicates price based on listing size
  price_per_bed = ifelse(beds > 0, price / beds, NA),

  #Indicator if listing is entire home, for price segmentation
  is_entire_home = ifelse(room_type == "Entire home/apt", 1, 0),

  #Log-transformed price to address skewness for sensitive distributions
  log_price = log1p(price)
) %>%

# Drop any rows that have missing values in the selected columns
drop_na() %>%

# Drop the raw amenities text column
select(-amenities)

# Random Forest cannot handle factors with more than 53 levels, so we group smaller neighborhoods into
opt_top_neighborhoods <- opt_LA_price %>%
  count(neighbourhood_cleansed, sort = TRUE) %>%
  slice(1:10) %>%
  pull(neighbourhood_cleansed) %>%
  as.character()

opt_LA_price <- opt_LA_price %>%
  mutate(neighbourhood_cleansed = fct_other(neighbourhood_cleansed, keep = opt_top_neighborhoods))

# Linear Regression to determine which variables are statistically
# significant

library(tidymodels)
set.seed(123)
opt_split <- initial_split(opt_LA_price, prop = 0.8)

```

```

train_data <- training(opt_split)
test_data <- testing(opt_split)

lm_model <- lm(price ~ room_type + neighbourhood_cleansed + number_of_reviews +
  review_scores_rating + minimum_nights + availability_365 + latitude +
  longitude + bedrooms + bathrooms + beds + amenities_count + price_per_bed +
  is_entire_home, data = train_data)

summary(lm_model)
##
## Call:
## lm(formula = price ~ room_type + neighbourhood_cleansed + number_of_reviews +
##     review_scores_rating + minimum_nights + availability_365 +
##     latitude + longitude + bedrooms + bathrooms + beds + amenities_count +
##     price_per_bed + is_entire_home, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2976.2   -64.0     1.1    50.4  11611.3
##
## Coefficients: (1 not defined because of singularities)
##              Estimate Std. Error
## (Intercept)   -22199.186955   1458.563682
## room_typeHotel room    37.171815    34.493949
## room_typePrivate room  -7.210831     5.226354
## room_typeShared room  -165.160142    28.986968
## neighbourhood_cleansedBeverly Hills  110.497479    18.978093
## neighbourhood_cleansedDowntown    -49.593177    19.549760
## neighbourhood_cleansedGlendale     2.768188    19.676928
## neighbourhood_cleansedHollywood   -47.335433    17.942441
## neighbourhood_cleansedLong Beach  -15.373069    16.600407
## neighbourhood_cleansedPasadena    26.412320    19.789432
## neighbourhood_cleansedSanta Monica -37.740526    18.097173
## neighbourhood_cleansedVenice     -13.144401    17.840719
## neighbourhood_cleansedWest Hollywood -29.157599    18.221615
## neighbourhood_cleansedOther     -11.871802    13.775950
## number_of_reviews    -0.031813     0.020870
## review_scores_rating   12.887221     4.984144
## minimum_nights       -0.327298     0.095787
## availability_365       0.027423     0.017428
## latitude             -96.359657    13.472919
## longitude            -213.147215    11.821508
## bedrooms              50.050038     3.334570
## bathrooms             136.508612     2.801895
## beds                  9.229064     1.924280
## amenities_count        0.288165     0.123402
## price_per_bed         1.031553     0.002632
## is_entire_home                NA          NA
##              t value      Pr(>|t|)
## (Intercept)   -15.220 < 0.0000000000000002 ***
## room_typeHotel room    1.078        0.281211
## room_typePrivate room  -1.380        0.167693
## room_typeShared room   -5.698    0.000000012320831 ***

```

```
## neighbourhood_cleansedBeverly Hills      5.822      0.000000005895562 ***
## neighbourhood_cleansedDowntown           -2.537      0.011196 *
## neighbourhood_cleansedGlendale            0.141      0.888123
## neighbourhood_cleansedHollywood           -2.638      0.008342 **
## neighbourhood_cleansedLong Beach          -0.926      0.354424
## neighbourhood_cleansedPasadena            1.335      0.182001
## neighbourhood_cleansedSanta Monica        -2.085      0.037043 *
## neighbourhood_cleansedVenice              -0.737      0.461275
## neighbourhood_cleansedWest Hollywood     -1.600      0.109579
## neighbourhood_cleansedOther               -0.862      0.388821
## number_of_reviews                        -1.524      0.127440
## review_scores_rating                      2.586      0.009727 **
## minimum_nights                           -3.417      0.000635 ***
## availability_365                          1.574      0.115610
## latitude                                 -7.152      0.000000000000886 ***
## longitude                                -18.030 < 0.0000000000000002 ***
## bedrooms                                  15.009 < 0.0000000000000002 ***
## bathrooms                                 48.720 < 0.0000000000000002 ***
## beds                                       4.796      0.000001630020101 ***
## amenities_count                           2.335      0.019545 *
## price_per_bed                             391.977 < 0.0000000000000002 ***
## is_entire_home                            NA          NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 268.1 on 18963 degrees of freedom
## Multiple R-squared:  0.9003, Adjusted R-squared:  0.9002
## F-statistic: 7136 on 24 and 18963 DF, p-value: < 0.0000000000000002
```

```
# Load libraries
library(ggplot2)
library(dplyr)

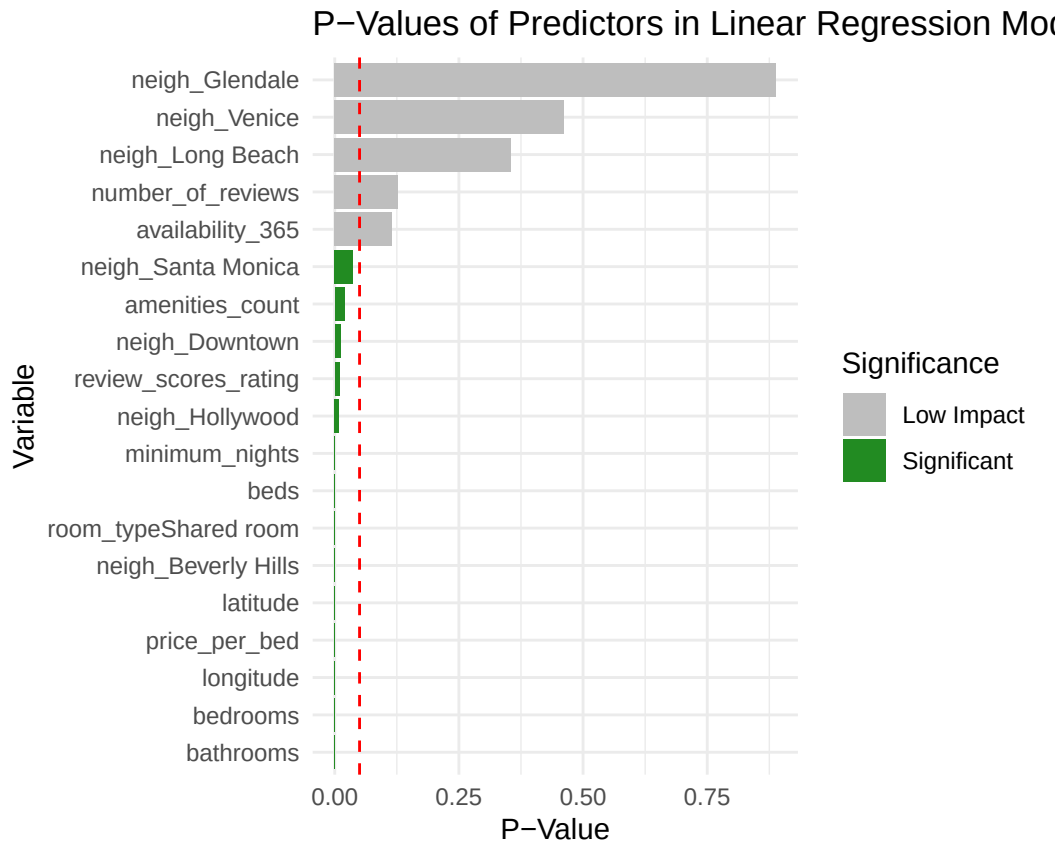
# Create the dataset
pval_data <- data.frame(Variable = c("bedrooms", "bathrooms", "beds", "review_scores_rating",
  "minimum_nights", "latitude", "longitude", "room_typeShared room",
  "neigh_Beverly Hills", "neigh_Downtown", "neigh_Hollywood", "neigh_Santa Monica",
  "amenities_count", "price_per_bed", "neigh_Glendale", "neigh_Long Beach",
  "neigh_Venice", "number_of_reviews", "availability_365"), p_value = c(0.0000000000000002,
  0.0000000000000002, 0.0000016, 0.009727, 0.000635, 0.000000000000886,
  0.0000000000000002, 0.0000000123, 0.00000000589, 0.0112, 0.0083, 0.037,
  0.0195, 0.0000000000000002, 0.8881, 0.3544, 0.4613, 0.1274, 0.1156))

# Add a significance flag
pval_data <- pval_data %>%
  mutate(Significance = ifelse(p_value < 0.05, "Significant", "Low Impact"))

# Sort for better plotting
pval_data <- pval_data %>%
  arrange(p_value)

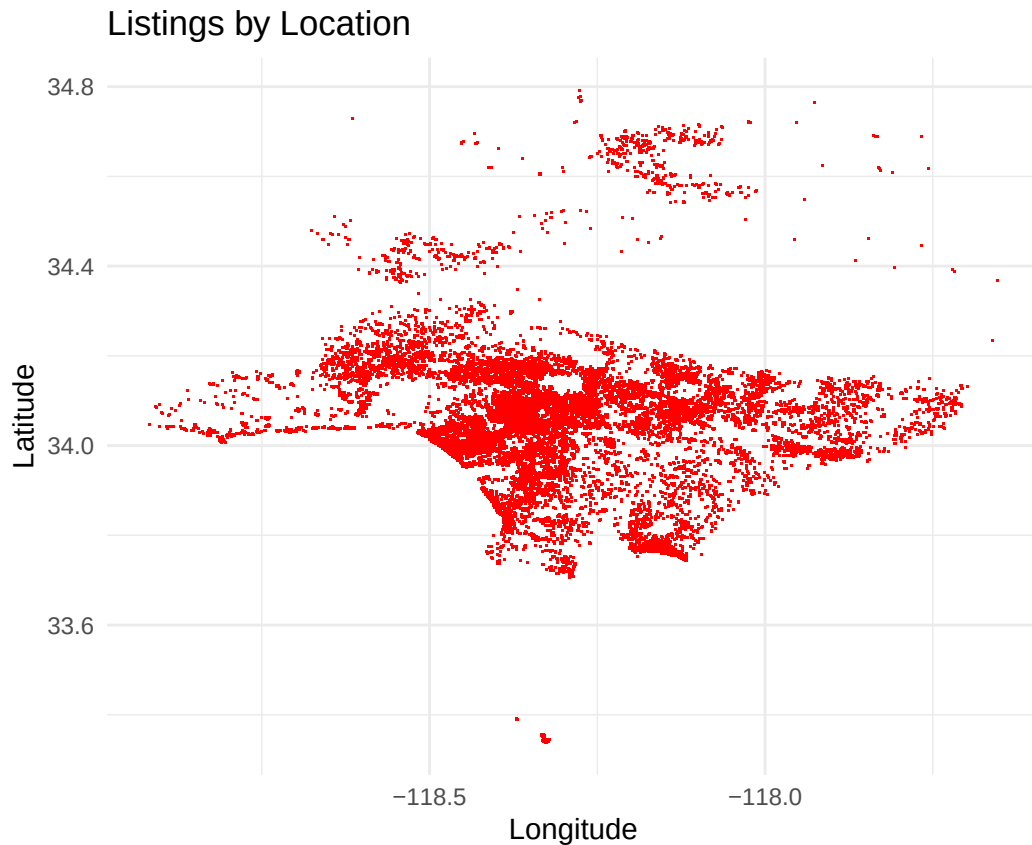
# Plot
ggplot(pval_data, aes(x = reorder(Variable, p_value), y = p_value, fill = Significance)) +
```

```
geom_col() + coord_flip() + geom_hline(yintercept = 0.05, linetype = "dashed",
color = "red") + labs(title = "P-Values of Predictors in Linear Regression Model",
x = "Variable", y = "P-Value") + scale_fill_manual(values = c(Significant = "forestgreen",
`Low Impact` = "gray")) + theme_minimal()
```



Map of rentals

```
ggplot(opt_LA_price, aes(x = longitude, y = latitude)) + geom_point(shape = ".",
color = "red") + labs(title = "Listings by Location", x = "Longitude",
y = "Latitude") + theme_minimal()
```

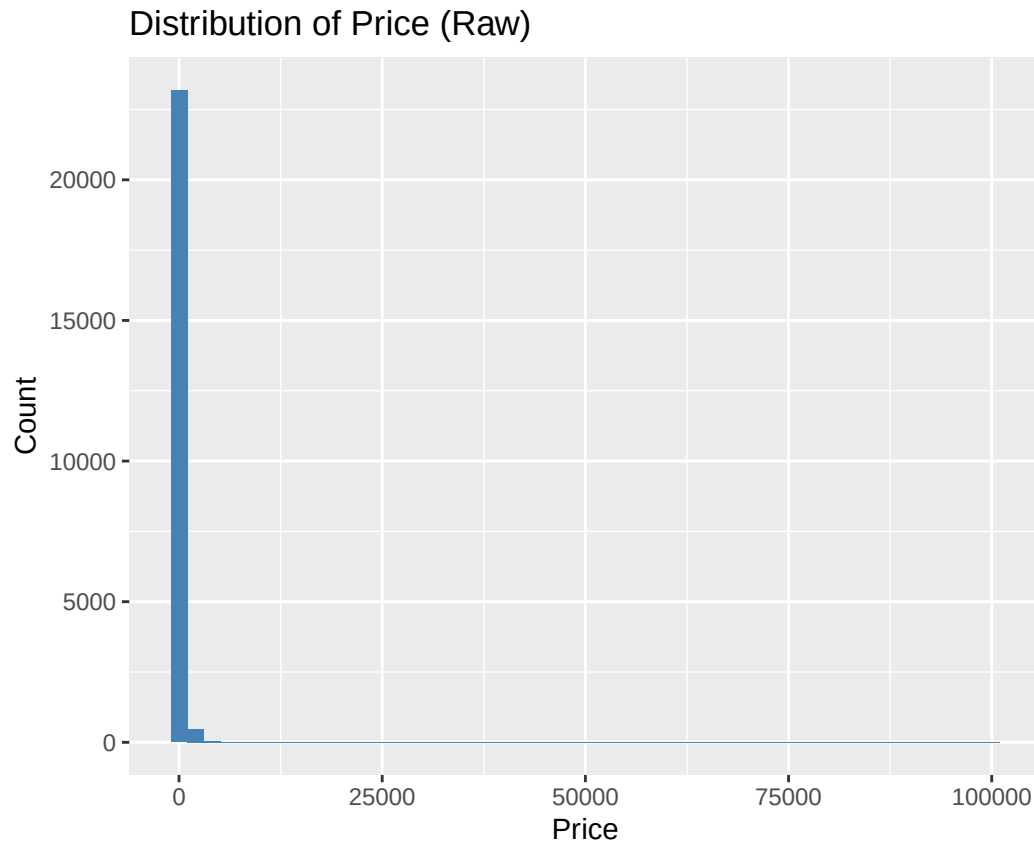


```
# Split the data into training and testing sets

# Use rsample's initial_split() to create an 80/20 train/test split
set.seed(123)
opt_split <- initial_split(opt_LA_price, prop = 0.8)

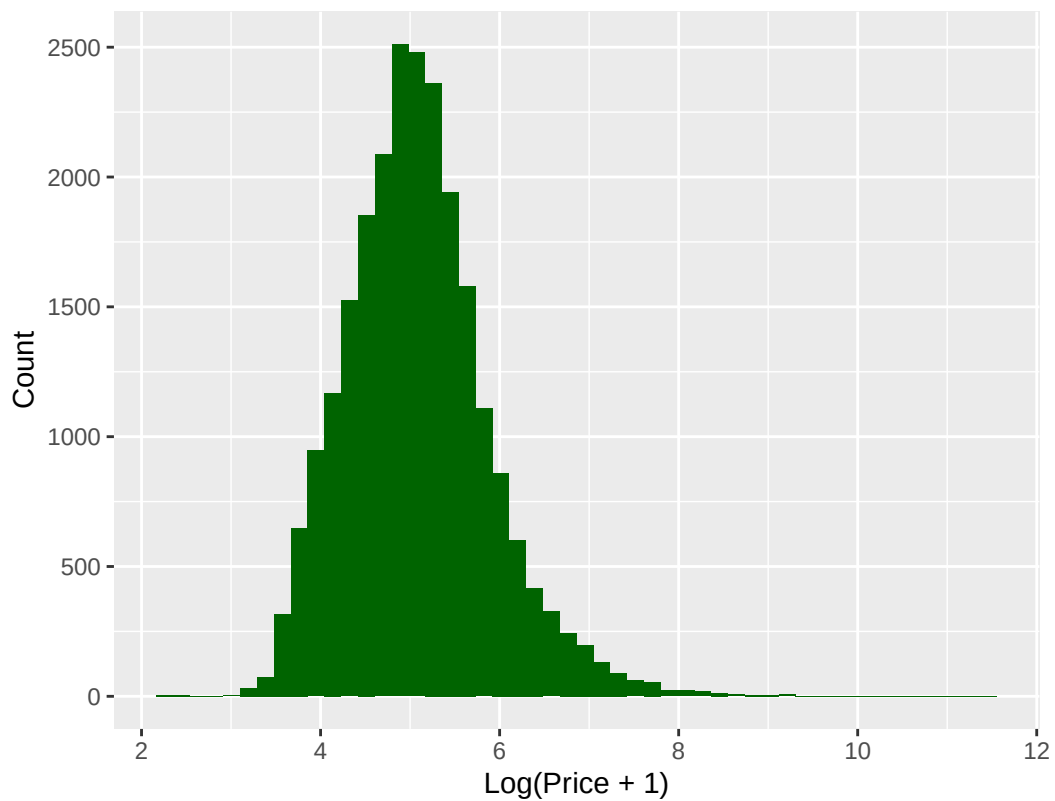
# Extract training and testing sets
opt_LA_train <- training(opt_split)
opt_LA_test <- testing(opt_split)

# Price distribution (before and after log transform)
ggplot(opt_LA_price, aes(x = price)) + geom_histogram(bins = 50, fill = "steelblue") +
  labs(title = "Distribution of Price (Raw)", x = "Price", y = "Count")
```



```
ggplot(opt_LA_price, aes(x = log_price)) + geom_histogram(bins = 50, fill = "darkgreen") +  
  labs(title = "Distribution of Price (Log-Transformed)", x = "Log(Price + 1)",  
        y = "Count")
```

Distribution of Price (Log-Transformed)



Model Building

Random Forest

```
# Train the Random Forest model

# Fit a random forrest regression model to predict nightly price
opt_rf_model <- randomForest(

  # Predict price using all other variables
  price ~ .,

  # Use training data only
  data = opt_LA_train,

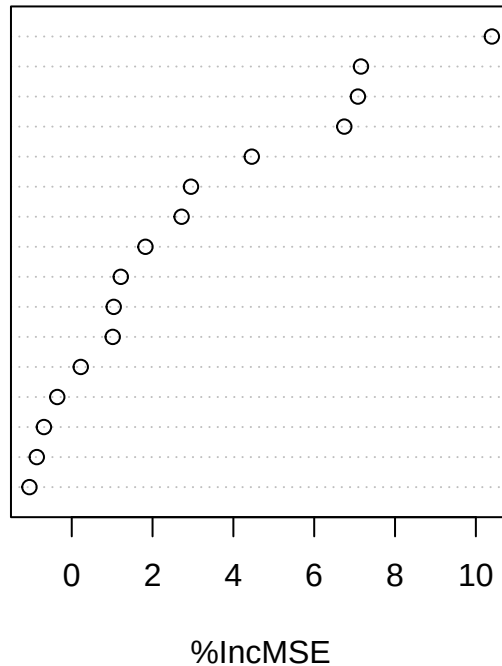
  # Number of trees in the forest
  ntree = 100,

  # Track feature importance for interpretation
  importance = TRUE
)

# Feature importance from random forest
varImpPlot(opt_rf_model, type = 1, main = "Feature Importance")
```

Feature Importance

log_price
bedrooms
price_per_bed
price_per_bedroom
bathrooms
beds
room_type
latitude
review_scores_rating
number_of_reviews
minimum_nights
neighbourhood_cleansed
amenities_count
longitude
is_entire_home
availability_365

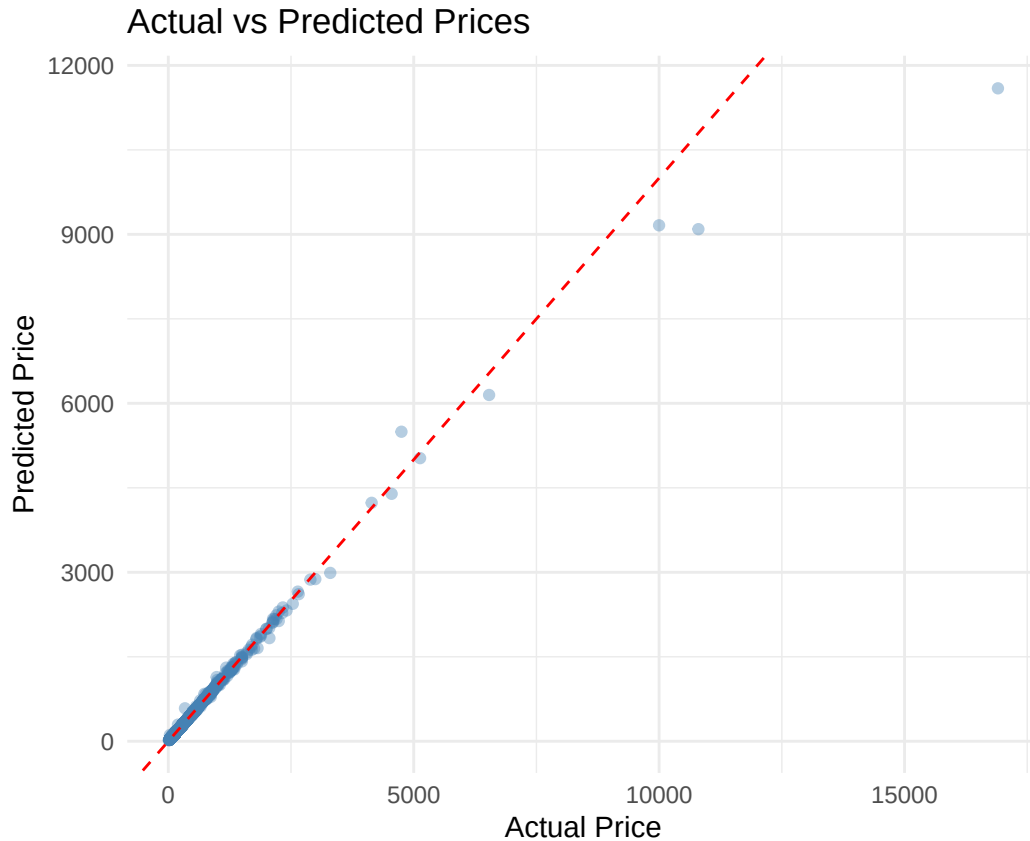


```
# Predict on the test data
predictions <- predict(opt_rf_model, newdata = opt_LA_test)

# Calculate performance metrics
mae <- mean(abs(predictions - opt_LA_test$price))
rmse <- sqrt(mean((predictions - opt_LA_test$price)^2))
r2 <- cor(predictions, opt_LA_test$price)^2

# Print the results
cat("MAE:", round(mae, 2), "\n")
## MAE: 4.05
cat("RMSE:", round(rmse, 2), "\n")
## RMSE: 83.49
cat("R2:", round(r2, 3), "\n")
## R2: 0.975

# Actual vs Predicted price scatterplot
ggplot(data.frame(actual = opt_LA_test$price, predicted = predictions),
  aes(x = actual, y = predicted)) + geom_point(alpha = 0.4, color = "steelblue") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  labs(title = "Actual vs Predicted Prices", x = "Actual Price", y = "Predicted Price") +
  theme_minimal()
```



Model Evaluation

MAE: 4.05 RMSE: 83.49 R2: 0.975

The R2 result is very high, and explains 97.5% of the variance seen in Airbnb prices. The MAE is relatively low and, like R2, suggests that the model performs well for most listings. The high RSME indicates that there are a few extreme-priced listings that produce large errors in the dataset. These listings may include luxury properties or otherwise unique offerings.

Price Optimization - Linear Regression Model

Since there are 3 objectives, all of which will require testing, training, and likely creation of similar data sets, all data groups created for linear regression price optimization will begin with “lm_”

```
# Use the cleaned dataset from earlier steps Assuming opt_LA_price is
# already cleaned and preprocessed

# Step 1: Train/test split
set.seed(123)

lm_sample_index <- sample(nrow(opt_LA_price), 0.8 * nrow(opt_LA_price))
lm_opt_train <- opt_LA_price[lm_sample_index, ]
lm_opt_test <- opt_LA_price[-lm_sample_index, ]
```

```
# Step 2: Fit a basic linear regression model
lm_model1 <- lm(price ~ room_type + neighbourhood_cleansed + number_of_reviews +
  review_scores_rating + minimum_nights + availability_365 + latitude +
  longitude + bedrooms + bathrooms + beds + amenities_count + price_per_bed +
  is_entire_home, data = lm_opt_train)
```

```
# Step 3: Predict and evaluate
```

```
lm_preds <- predict(lm_model, newdata = lm_opt_test)
```

```
# Step 4: Evaluate
```

```
actual <- lm_opt_test$price
mae <- mean(abs(actual - lm_preds))
rmse <- sqrt(mean((actual - lm_preds)^2))
r2 <- cor(actual, lm_preds)^2

cat("MAE:", round(mae, 2), "\n")
## MAE: 93.71
cat("RSME:", round(rmse, 2), "\n")
## RSME: 283.63
cat("R-squared:", round(r2, 3), "\n")
## R-squared: 0.59
```

Model Evaluation

MAE = \$93.71. On average, our predicted prices are off by \$93.71. This is a moderate error for price prediction.

RMSE = \$283.63. There are some large prediction errors(possibly due to outliers or high-price listings). RMSE is more sensitive to these than MAE.

R-Squared = 0.59. The model explains 59% of the variation in listing prices. That means over half of the pricing behavior is captured by our input features, but 41% is unexplained. # Comparison to Random Forest Random Forest Results: MAE = \$4.05 RMSE = \$83.49 R-Squared = 0.975

Random Forest clearly outperforms linear regression in both accuracy and predictive power. Linear regression is easier to interpret, but sacrifices precision. The gap in RMSE suggests that linear regression struggles with more complex price dynamics - especially for high-end listings or where non-linear relationships exist.

Revenue Forecasting

Estimate annual revenue per listing using reported attributes of each listing for hosts to make informed decisions about setting prices and availability on their properties. Estimate expected future revenue for listings using pricing, availability, and booking history.

Since there are 3 objectives, all of which will require testing, training, and likely creation of similar data sets, all data groups created for revenue forecasting will begin with “rev_”.

Data Preparation

```
# Select relevant columns and clean them for modeling

rev_LA_avail <- LA_listings %>%
  select(price, room_type, neighbourhood_cleansed, number_of_reviews, review_scores_rating,
         minimum_nights, availability_365, latitude, longitude, amenities,
         bedrooms, bathrooms, beds) %>%
  mutate(
    # Convert price string to numeric
    price = as.numeric(gsub("$", "", price)),

    # Create proxy for bookings, allowing number of reviews to stand in for bookings if no booking data
    # This assumes a correlation between reviews and bookings, which likely underestimates revenue for n
    proxy_bookings = number_of_reviews,

    # Create revenue variable and log-transformed revenue
    revenue = price * proxy_bookings,
    log_revenue = log1p(revenue),

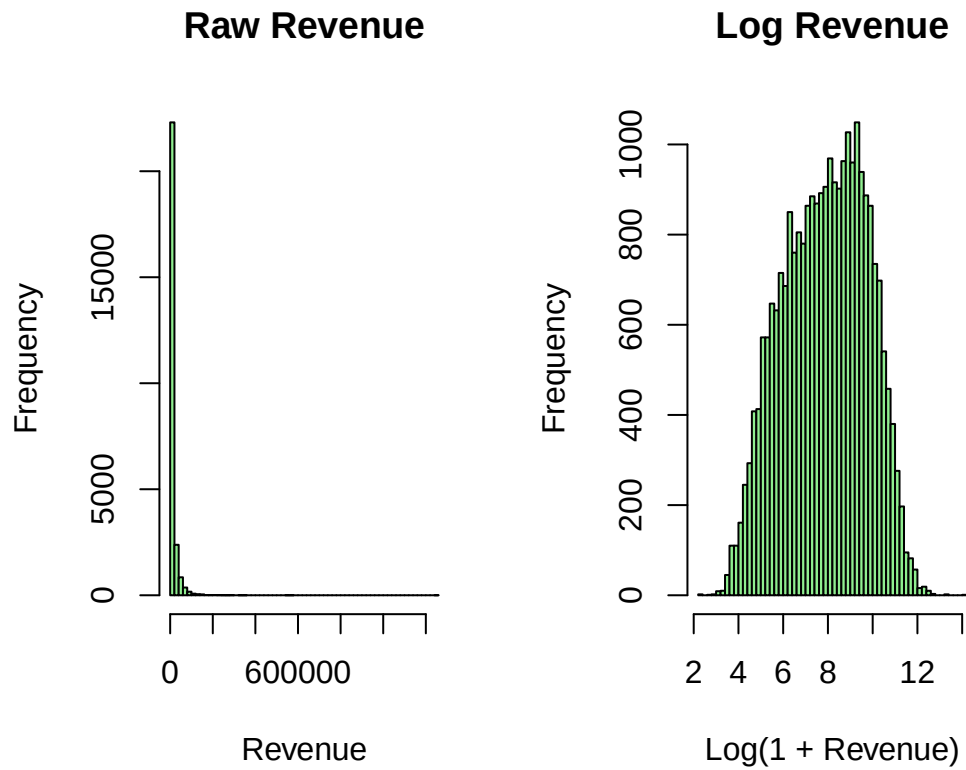
    # Isolate availability by singular days
    availability_rate = availability_365 / 365,

    # Convert categorical variables to factors
    room_type = as.factor(room_type),
    neighbourhood_cleansed = as.factor(neighbourhood_cleansed),

    # Convert bedroom/bathroom/bed counts to numeric
    bedrooms = as.numeric(bedrooms),
    bathrooms = as.numeric(bathrooms),
    beds = as.numeric(beds),

    # Create a new numeric feature: count of amenities
    amenities_count = str_count(amenities, ",") + 1
  ) %>%
  select(-amenities) %>% # Drop raw amenities text
  drop_na()

par(mfrow = c(1, 2))
hist(rev_LA_avail$revenue, main = "Raw Revenue", xlab = "Revenue", breaks = 50, col = "lightgreen")
hist(rev_LA_avail$log_revenue, main = "Log Revenue", xlab = "Log(1 + Revenue)", breaks = 50, col = "lightgreen")
```



```
# Create an 80/20 train/test split

rev_split <- initial_split(rev_LA_avail, prop = 0.8)

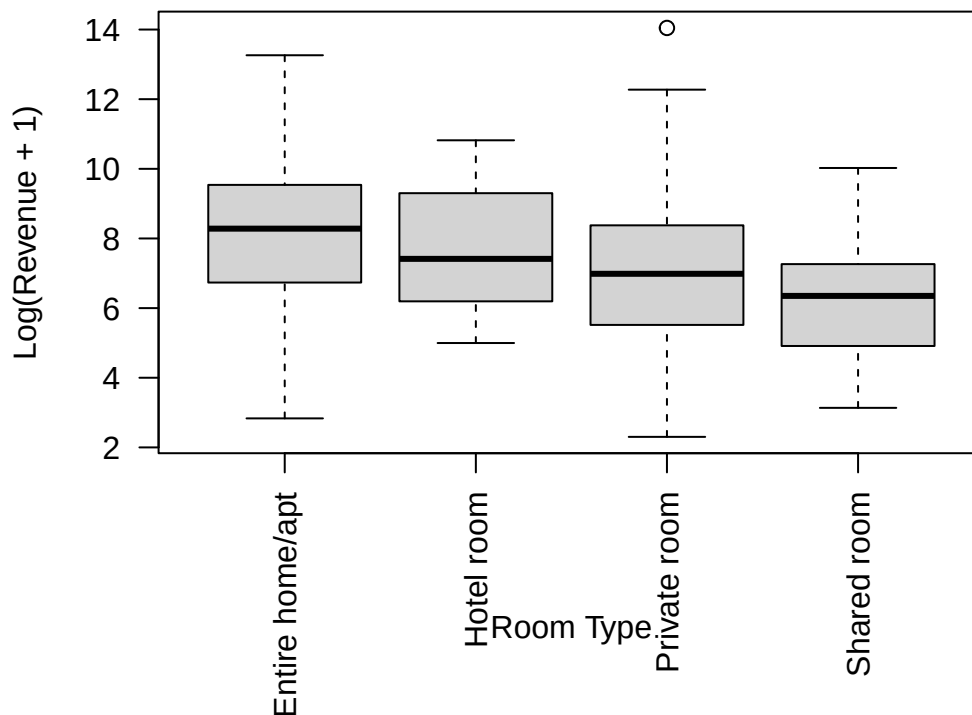
# Extract training and testing sets
rev_train <- training(rev_split)
rev_test  <- testing(rev_split)

par(mar = c(7, 4, 4, 2))

# Boxplot
boxplot(log_revenue ~ room_type, data = rev_LA_avail, col = "lightgray",
        main = "Log Revenue by Room Type", xlab = "", ylab = "Log(Revenue + 1)",
        las = 2)

mtext("Room Type", side = 1, line = 4)
```

Log Revenue by Room Type



Data Engineering

```
# We did not learn model.matrix in class, however, it was covered in
# BDS's section on regression and model selection in chapter 2

# Create revenue training and testing matrices
rev_train_matrix <- model.matrix(~. - 1, data = rev_train %>%
  select(-log_revenue))
rev_train_target <- rev_train$log_revenue

rev_test_matrix <- model.matrix(~. - 1, data = rev_test %>%
  select(-log_revenue))
rev_test_target <- rev_test$log_revenue
```

Forecasting Models

Elastic Net

```
# Fit by using cross-validation across multiple alpha values
set.seed(123)
```

```

enet_mod <- cva.glmnet(x = rev_train_matrix, y = rev_train_target, alpha = seq(0,
  1, length = 11), family = "gaussian")

# Helper function to find the alpha with lowest cross-validation
# error
rev_get_alpha <- function(fit) {
  alpha <- fit$alpha
  error <- sapply(fit$modlist, function(mod) {
    min(mod$cvm)
  })
  alpha[which.min(error)]
}

# Helper function to extract alpha, lambda.min, lambda.1se, and error
rev_get_model_params <- function(fit) {
  alpha <- fit$alpha
  lambdaMin <- sapply(fit$modlist, `[`, "lambda.min")
  lambdaSE <- sapply(fit$modlist, `[`, "lambda.1se")
  error <- sapply(fit$modlist, function(mod) {
    min(mod$cvm)
  })
  best <- which.min(error)
  data.frame(alpha = alpha[best], lambdaMin = lambdaMin[best], lambdaSE = lambdaSE[best],
    error = error[best])
}

# Identify best alpha and lambda values
rev_best_alpha <- rev_get_alpha(enet_mod)
print(paste("Best alpha:", rev_best_alpha))
## [1] "Best alpha: 0.1"

rev_model_params <- rev_get_model_params(enet_mod)
print(rev_model_params)
##   alpha lambdaMin lambdaSE   error
## 1   0.1 0.04584175 0.2446436 1.352027

# Extract best model using optimal alpha
rev_best_model <- enet_mod$modlist[[which(enet_mod$alpha == rev_best_alpha)]]

# Predict log revenue using the best model and optimal lambda
rev_enet_predictions <- predict(rev_best_model, s = rev_model_params$lambdaMin,
  newx = rev_test_matrix)

# Evaluate model performance on the test set
rev_enet_mae <- mean(abs(rev_enet_predictions - rev_test_target))
rev_enet_rmse <- sqrt(mean((rev_enet_predictions - rev_test_target)^2))
rev_enet_r2 <- cor(rev_enet_predictions, rev_test_target)^2

# Output performance results
cat("Elastic Net (best alpha =", rev_best_alpha, ") - MAE:", round(rev_enet_mae,
  2), "\n")
## Elastic Net (best alpha = 0.1 ) - MAE: 0.94
cat("Elastic Net (best alpha =", rev_best_alpha, ") - RMSE:", round(rev_enet_rmse,

```

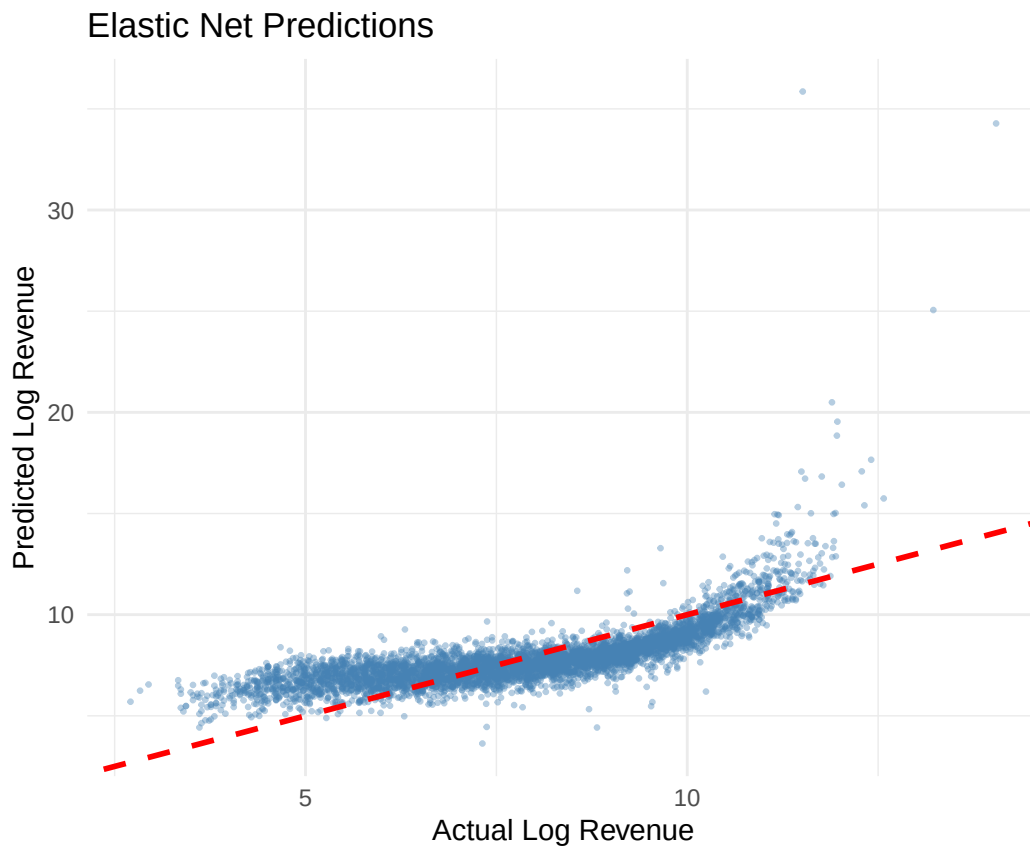
```

2), "\n")
## Elastic Net (best alpha = 0.1 ) - RMSE: 1.24
cat("Elastic Net (best alpha =", rev_best_alpha, ") - R²:", round(rev_enet_r2,
3), "\n")
## Elastic Net (best alpha = 0.1 ) - R²: 0.556

rev_plot_df <- data.frame(actual = as.numeric(rev_test_target), predicted = as.numeric(rev_enet_predict))

ggplot(rev_plot_df, aes(x = actual, y = predicted)) + geom_point(color = "steelblue",
  alpha = 0.4, size = 0.5) + geom_abline(slope = 1, intercept = 0, color = "red",
  linetype = "dashed", size = 1) + labs(title = "Elastic Net Predictions",
  x = "Actual Log Revenue", y = "Predicted Log Revenue") + theme_minimal()

```



Model Evaluation

MAE = 0.93. On average, our predicted log revenues are off by 0.93. This is a moderate error in the log scale, indicating reasonable but imperfect accuracy.

RMSE = 1.21. There are some larger prediction errors (perhaps due to extreme revenue values or outliers). RMSE is more sensitive to these than MAE.

R-Squared = 0.579. The model explains 57.9% of the variation in log revenue. This means just over half of the revenue behavior is captured by our input features, while 42.1% remains unexplained.

Since we are unable to see actual revenue per listing, it can be assumed that revenue is highly skewed based on seasonality and dynamic pricing.

End: Revenue forecasting

Market Segmentation

General Segmentation

The following lines are the first round of market segmenting through clustering models with “general” signifiers. These are taken as simple features that are standard across Airbnbs.

Data Preperation

```
# Select relevant columns and clean them for modeling, including both features of the property and indi
mktseg_cluster_data_general <- LA_listings %>%
  select(price, bedrooms, bathrooms, beds, review_scores_rating, availability_365, amenities) %>%
  mutate(

    # Convert price to numeric by removing formatting
    price = as.numeric(gsub("$", "", price)),

    bedrooms = as.numeric(bedrooms),
    bathrooms = as.numeric(bathrooms),
    beds = as.numeric(beds),
    review_scores_rating = as.numeric(review_scores_rating),
    availability_365 = as.numeric(availability_365),
    amenities_count = str_count(amenities, ",") + 1
  ) %>%

  #Now that amenities are counted, we drop the actual amenities inputs
  select(-amenities) %>%
  drop_na() %>%

#Sampling ~10% of data set

slice(1:4500)

#Scale variables so that they contribute equally, even when initially measured by different units (pric
mktseg_cluster_data_scaled_general <- scale(mktseg_cluster_data_general)
```

Modeling

```
# Elbow Method to Choose Optimal K

wss <- function(k) {
  kmeans(mktseg_cluster_data_scaled_general, centers = k, nstart = 25)$tot.withinss
```

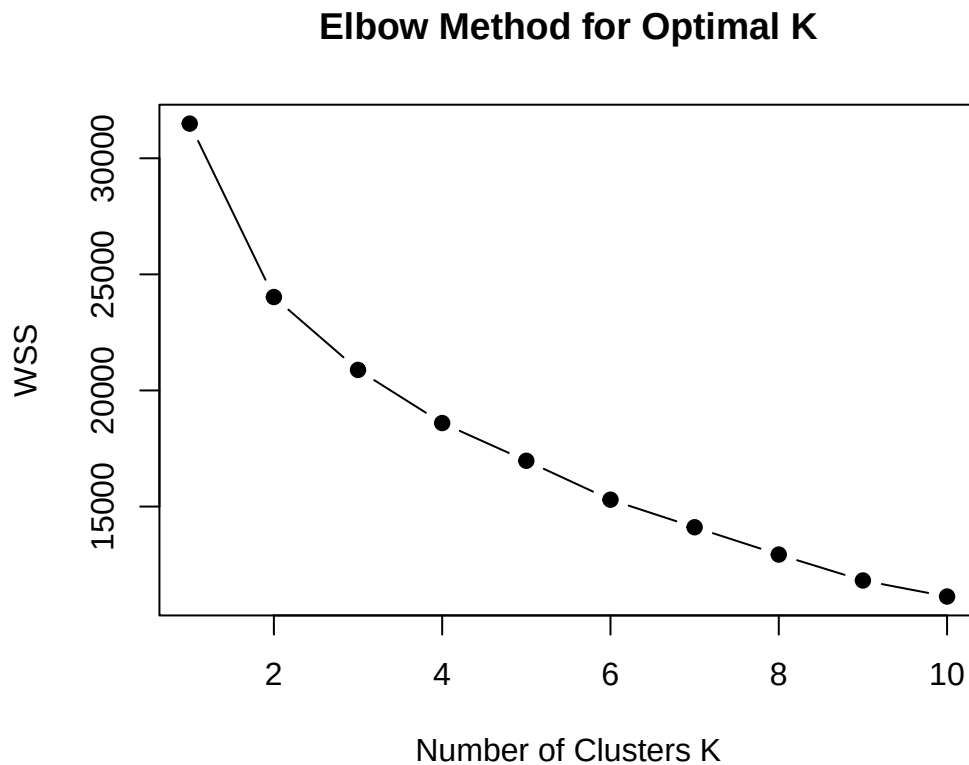
```

}
wss_values <- map_dbl(1:10, wss)

plot(1:10, wss_values, type = "b", pch = 19, xlab = "Number of Clusters K",
     ylab = "WSS", main = "Elbow Method for Optimal K")

```

K-means



```

# Run PCA on the scaled general data
mktseg_pca_general <- prcomp(mktseg_cluster_data_scaled_general, scale = FALSE)

# Perform K-means clustering on scaled general data
kmeans_general_result <- kmeans(mktseg_cluster_data_scaled_general, centers = 8,
                                nstart = 25)

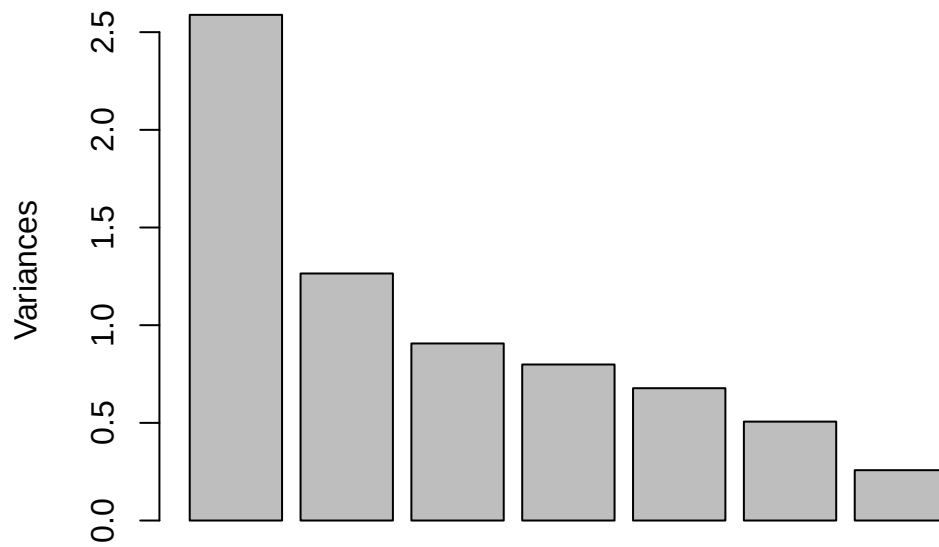
# Add cluster labels to original unscaled general data
mktseg_clustered_general <- mktseg_cluster_data_general %>%
  mutate(segment = as.factor(kmeans_general_result$cluster))

# Scree plot of PCA
screeplot(mktseg_pca_general, main = "Scree Plot of PCA")

```

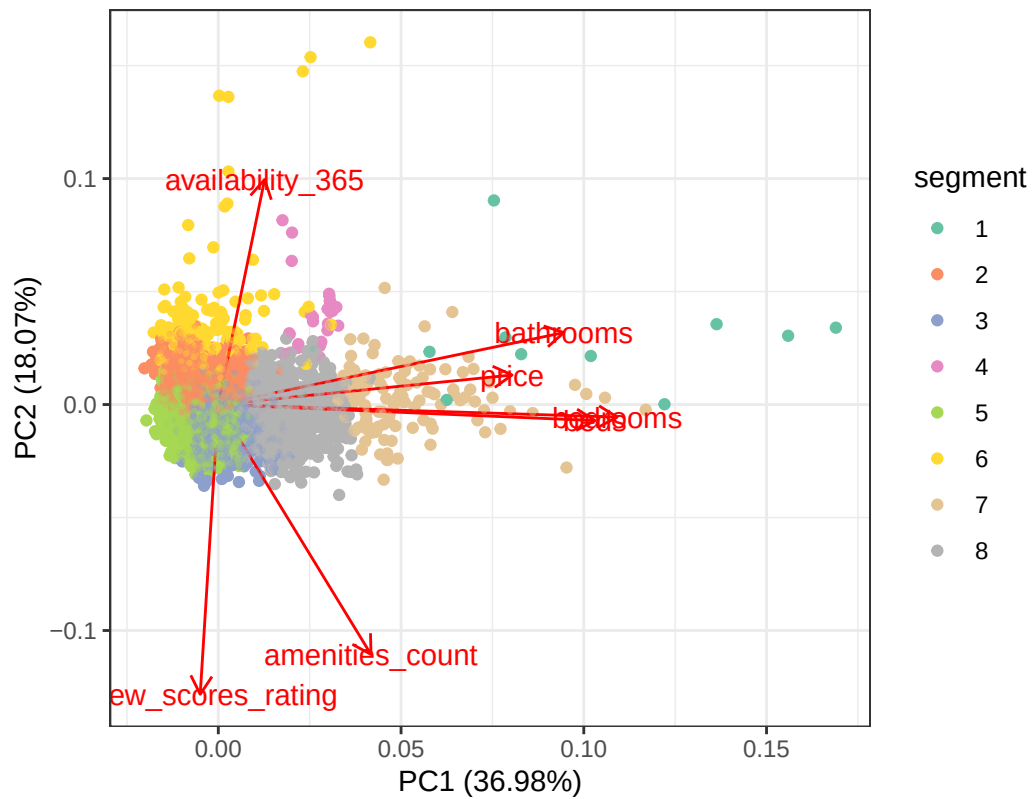
PCA

Scree Plot of PCA



```
# PCA biplot with cluster segments
autoplot(mktseg_pca_general, data = mktseg_clustered_general, colour = "segment",
  loadings = TRUE, loadings.label = TRUE, loadings.label.size = 4) +
  geom_point(aes(color = segment), alpha = 0.4, size = 0.5) + scale_color_brewer(palette = "Set2") +
  theme_bw() + ggtitle("PCA Biplot with K-Means Segment Labels")
```

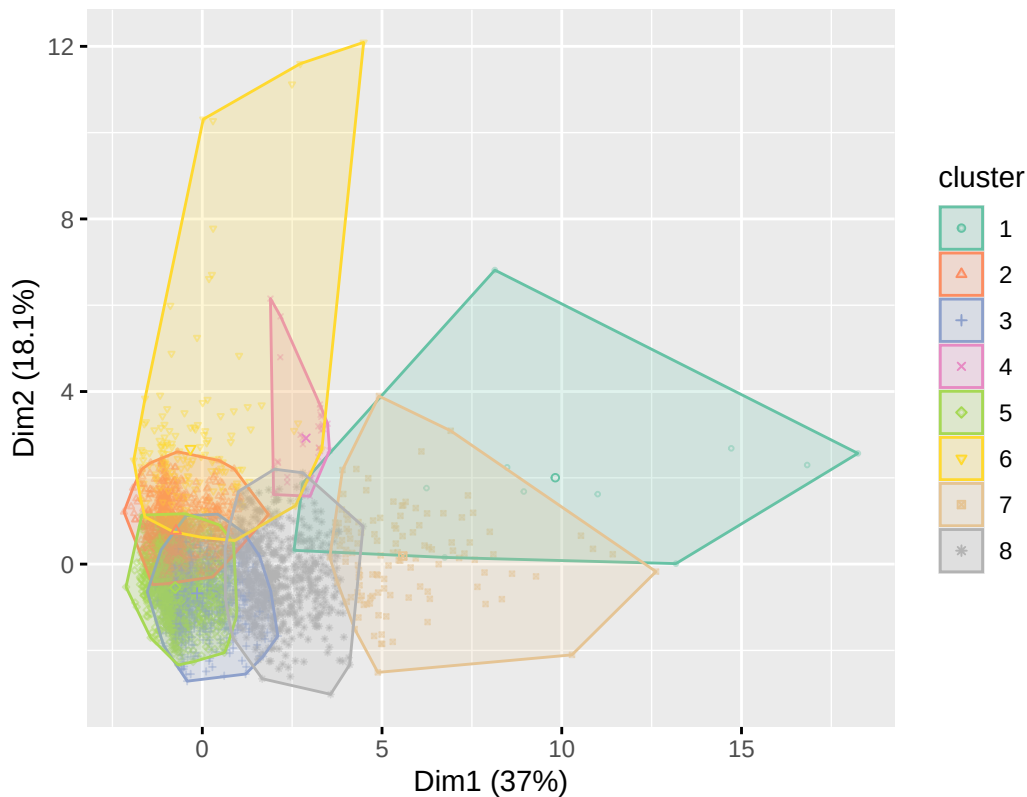
PCA Biplot with K-Means Segment Labels



```
# Add the resulting cluster assignments as a new variable, so now each
# row belongs to one of the k (8) market segments (labeled as 1
# through 8)

fviz_cluster(kmeans_general_result, data = mktseg_cluster_data_scaled_general,
  main = "K=8 Means Clustering on Airbnb Data", palette = "Set2", geom = "point",
  pointsize = 0.5, alpha = 0.4, show.clust.cent = TRUE, repel = TRUE)
```

K=8 Means Clustering on Airbnb Data



```
# Track row IDs for consistent matching with clustering results
mktseg_clustered_general <- mktseg_cluster_data_general %>%
  mutate(segment = as.factor(kmeans_general_result$cluster))

# Summarize the characteristics of each segment, helps interpret how
# different segments differ in key features (e.g., price, size,
# amenities)
mktseg_segment_summary_general <- mktseg_clustered_general %>%
  group_by(segment) %>%
  summarise(avg_price = mean(price), avg_bedrooms = mean(bedrooms), avg_bathrooms = mean(bathrooms),
    avg_beds = mean(beds), avg_rating = mean(review_scores_rating,
      na.rm = TRUE), avg_amenities = mean(amenities_count), avg_availability = mean(availability_365))

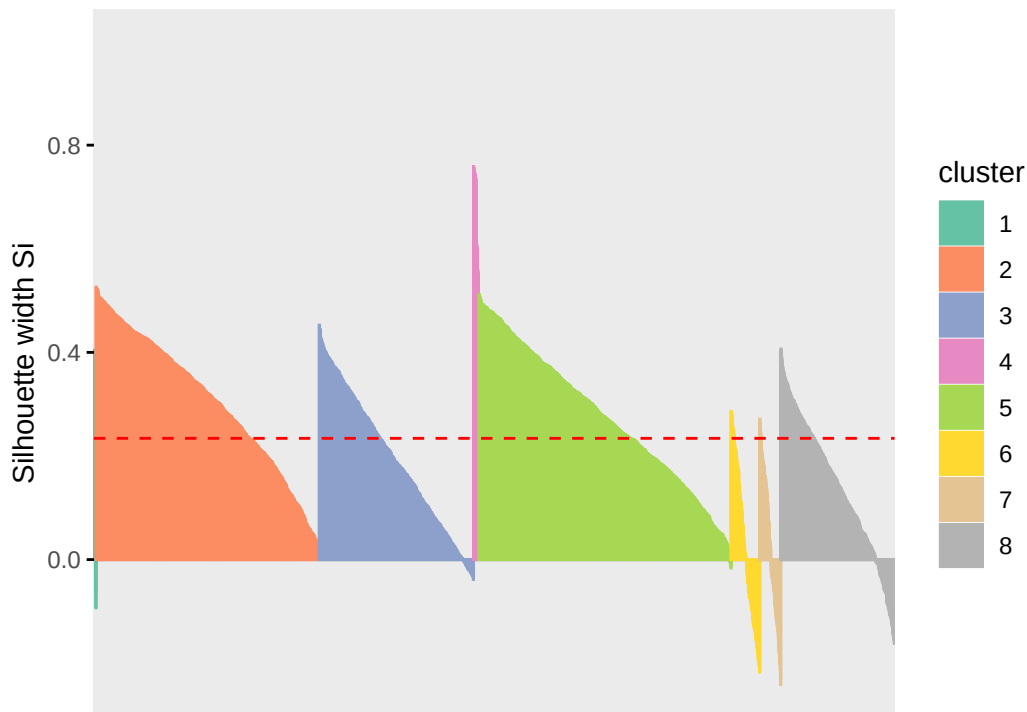
# View the summary table of cluster profiles
print(mktseg_segment_summary_general)
## # A tibble: 8 x 8
##   segment avg_price avg_bedrooms avg_bathrooms avg_beds avg_rating
##   <fct>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 1        7387.        4.58        5.46        4.83        4.60
## 2 2         134.        1.04        1.14        1.40        4.81
## 3 3         173.        1.24        1.17        1.74        4.88
## 4 4          46.2        0.906       9.53        1.88        4.51
## 5 5         136.        1.05        1.12        1.43        4.85
## 6 6         200.        1.30        1.31        1.80        4.06
## 7 7       1274.        4.80        4.51        7.24        4.82
```

```
## 8 8          427.          3.13          2.31          3.99          4.84
## # i 2 more variables: avg_amenities <dbl>, avg_availability <dbl>

# Compute silhouette scores
silhouette_score_general <- silhouette(kmeans_general_result$cluster, dist(mktseg_cluster_data_scaled_g

# Visualize silhouette plot
fviz_silhouette(silhouette_score_general, palette = "Set2", ggtheme = theme_minimal(),
  print.summary = TRUE)
##   cluster size ave.sil.width
## 1         1   12         0.16
## 2         2 1254         0.30
## 3         3  867         0.19
## 4         4   32         0.64
## 5         5 1416         0.27
## 6         6  162         0.03
## 7         7  118         0.03
## 8         8  639         0.14
```

Clusters silhouette plot
Average silhouette width: 0.23



Understanding silhouette scores: $\sim 0.5+$ = strong structure $\sim 0.25-0.5$ = fair structure $\sim 0.0-0.25$ = weak structure, potentially overlapping clusters < 0 = potential misclassification

General clustering results:

Average silhouette score of 0.22 suggests a weak to moderate structure, which provides some context but many listings still have decision boundaries that are poorly matched.

1: -0.05 2: 0.31 3: 0.24 4: 0.15 5: 0.29 6: 0.23 7: 0.15 8: 0.01

Cluster 1 has a negative and small silhouette, suggesting that the cluster is made of listings that had no where else to group in with. Cluster 8 also is deeply negative, isolated by it's high prices, but does not fit in another boundary well enough.

Clusters 2, 3, 5, and 6 show better separation. 3 is defined by high-performiing listings that are smaller properties (maybe apartments, guest suites) run by experienced hosts that offer many amentities. 3 has some overlap with clusters 5 and 6. 5 homes the most budget-friendly listings that have low availability and moderate amount of amentities without sacrificing quality (Average rating is 4.84 per property). 6 also holds high rated properties (average = 4.75) but has slightly fewer amentities for a higher price. The length of stay and the consistent high rating for lower amentities may indicate greater bedrooms and bathrooms and long-term or flexible stay options.

Cluster 2 is the most fit cluster and represents highly available, low-quality (average = 2.4 rating) properties. If a host is in cluster 2, they may have properties in poor condition, treat guests worse than expected, and have fewer amenities.

Amenities Segmentation

Other opportunities to cluster data to further understand groupings are present, including amenities offered and location. An example of what these models could look like are included in the remaining document but are not fully evaluated or refined and are left as a place for future projects to build from.

Data Preparation

```
# Engineer binary amenities
mktseg_cluster_data_amenities <- LA_listings %>%
  select(price, bedrooms, bathrooms, beds, review_scores_rating, availability_365, amenities) %>%
  mutate(

    #Includes initial variables as mktseg_cluster_data_general
    price = as.numeric(gsub("$,", "", price)),
    bedrooms = as.numeric(bedrooms),
    bathrooms = as.numeric(bathrooms),
    beds = as.numeric(beds),
    review_scores_rating = as.numeric(review_scores_rating),
    availability_365 = as.numeric(availability_365),
    amenities_count = str_count(amenities, ",") + 1,

    has_wifi = grepl("Wifi", amenities, ignore.case = TRUE),
    has_kitchen = grepl("Kitchen", amenities, ignore.case = TRUE),
    has_air_conditioning = grepl("Air conditioning", amenities, ignore.case = TRUE),
    has_external_camera = grepl("Exterior security cameras on property", amenities, ignore.case = TRUE),
    has_washer = grepl("Washer", amenities, ignore.case = TRUE),
    has_tv = grepl("TV", amenities, ignore.case = TRUE),
    has_coffee_maker = grepl("Coffee maker", amenities, ignore.case = TRUE),
    has_free_parking = grepl("Free parking", amenities, ignore.case = TRUE),
    has_private_pool = grepl("Private pool", amenities, ignore.case = TRUE)
  ) %>%
```

```

select(-amenities) %>%

drop_na()

# Scale features for clustering (excluding amenities_count for now)
mktseg_cluster_data_scaled_amenities <- scale(
  mktseg_cluster_data_amenities %>% select(-amenities_count)
)

```

Modeling

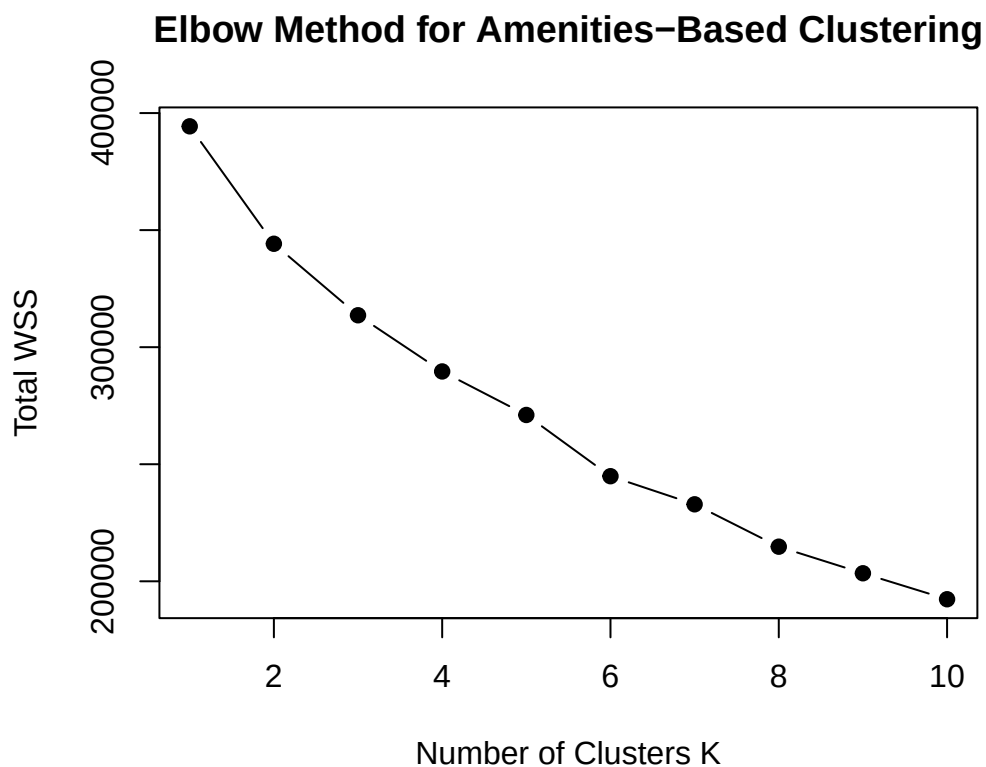
```

# Elbow method to select optimal K
wss_amenities <- function(k) {
  kmeans(mktseg_cluster_data_scaled_amenities, centers = k, nstart = 25)$tot.withinss
}
wss_values_amenities <- map_dbl(1:10, wss_amenities)

plot(1:10, wss_values_amenities, type = "b", pch = 19, xlab = "Number of Clusters K",
     ylab = "Total WSS", main = "Elbow Method for Amenities-Based Clustering")

```

K-means



```
# Apply K-means clustering (adjust k as needed)
```

```
# For future exploration: Be sure to convert amenities to binary  
# values, then suggested code may be used as a guide.
```

```
Track row IDs to join segment labels back mktseg_clustered_amenities <- mktseg_cluster_data_amenities  
%>% mutate(row_id = as.integer(rownames(.)), segment = as.factor(mktseg_kmeans_result_amenities$cluster))  
mktseg_LA_listings_amenities <- LA_listings %>% mutate(row_id = row_number())  
mktseg_LA_segmented_amenities <- mktseg_LA_listings_amenities %>% left_join(select(mktseg_clustered_amenities,  
row_id, segment, amenities_count), by = "row_id") %>% select(-row_id)
```

Summary of cluster profiles

```
mktseg_segment_summary_amenities <- mktseg_LA_segmented_amenities %>% filter(!is.na(segment))  
%>% group_by(segment) %>% summarise( avg_price = mean(as.numeric(gsub("[$,]", "", price))),  
avg_bedrooms = mean(as.numeric(bedrooms)), avg_bathrooms = mean(as.numeric(bathrooms)),  
avg_beds = mean(as.numeric(beds)), avg_rating = mean(as.numeric(review_scores_rating), na.rm =  
TRUE), avg_amenities = mean(amenities_count), avg_availability = mean(as.numeric(availability_365))  
)
```

View the summary table of cluster profiles

```
print(mktseg_segment_summary_amenities)
```

Silhouette score

```
silhouette_scores_amenities <- silhouette(mktseg_kmeans_result_amenities$cluster, dist(mktseg_cluster_data_scaled_amenities))  
plot(silhouette_scores_amenities)
```

PCA

PCA on the scaled amenities clustering data

```
mktseg_pca_amenities <- prcomp(mktseg_cluster_data_scaled_amenities, scale = FALSE)
```

Scree plot on the scaled amenities clustering data

```
screeplot(mktseg_pca_amenities, main = "Scree Plot for Amenities PCA")
```

ggplot on the scaled amenities clustering data

```
autoplot(mktseg_pca_amenities, data = mktseg_clustered_amenities, colour = 'segment', loadings =  
TRUE, loadings.label = TRUE, loadings.label.size = 4) + theme_bw() + ggtitle("PCA Biplot with Cluster  
Labels (Amenities-Based)")
```

Location Segmentation

Other opportunities to cluster data to further understand groupings are present, including amenities offered and location. An example of what these models could look like are included in the remaining document but are not fully evaluated or refined and are left as a place for future projects to build from.

Data Preparation

```
# Add row_id early so we can use it later for safe joins
mktseg_cluster_data_location <- LA_listings %>%
  mutate(row_id = row_number(), is_downtown = neighbourhood_cleansed %in%
    c("Downtown", "Koreatown", "Arts District", "South Park"), is_westside = neighbourhood_cleansed %in%
    c("Santa Monica", "Venice", "Marina del Rey", "West LA"), is_eastside = neighbourhood_cleansed %in%
    c("Echo Park", "Silver Lake", "Los Feliz"), is_valley = neighbourhood_cleansed %in%
    c("North Hollywood", "Studio City", "Sherman Oaks", "Van Nuys")) %>%
  select(row_id, price, bedrooms, bathrooms, beds, review_scores_rating,
    availability_365, is_downtown, is_westside, is_eastside, is_valley) %>%
  mutate(price = as.numeric(gsub("$", "", price)), bedrooms = as.numeric(bedrooms),
    bathrooms = as.numeric(bathrooms), beds = as.numeric(beds), review_scores_rating = as.numeric(review_scores_rating),
    availability_365 = as.numeric(availability_365)) %>%
  drop_na()
```

```
# Scale data
mktseg_cluster_data_scaled_location <- scale(mktseg_cluster_data_location %>%
  select(-row_id))
```

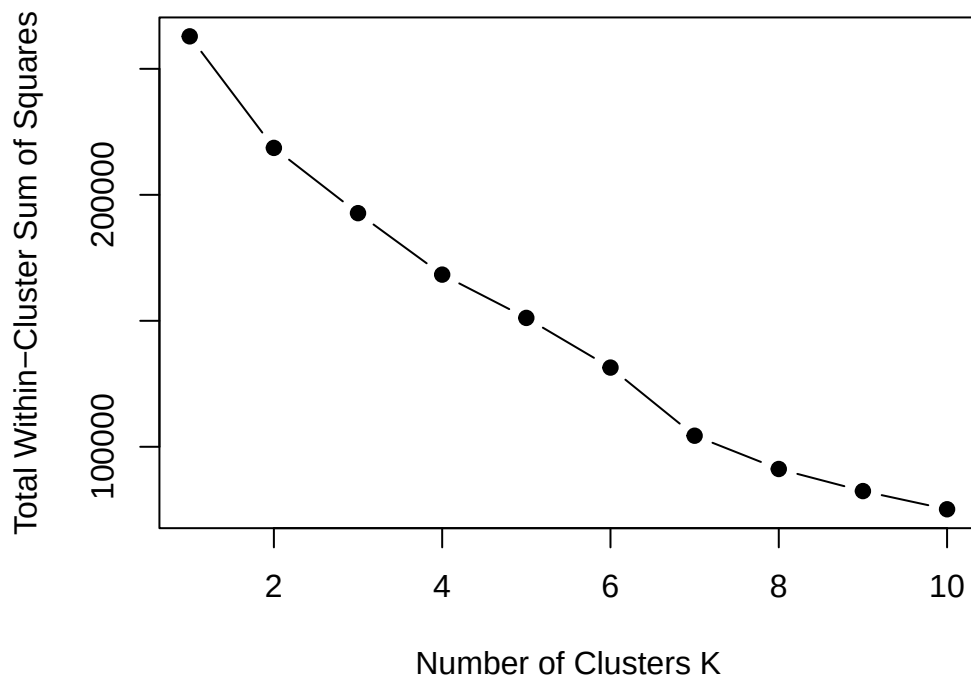
Modeling

```
wss_location <- function(k) {
  kmeans(mktseg_cluster_data_scaled_location, centers = k, nstart = 25)$tot.withinss
}
wss_values_location <- map_dbl(1:10, wss_location)

plot(1:10, wss_values_location, type = "b", pch = 19, xlab = "Number of Clusters K",
  ylab = "Total Within-Cluster Sum of Squares", main = "Elbow Method for Optimal K (Location Proxies)")
```

K-means

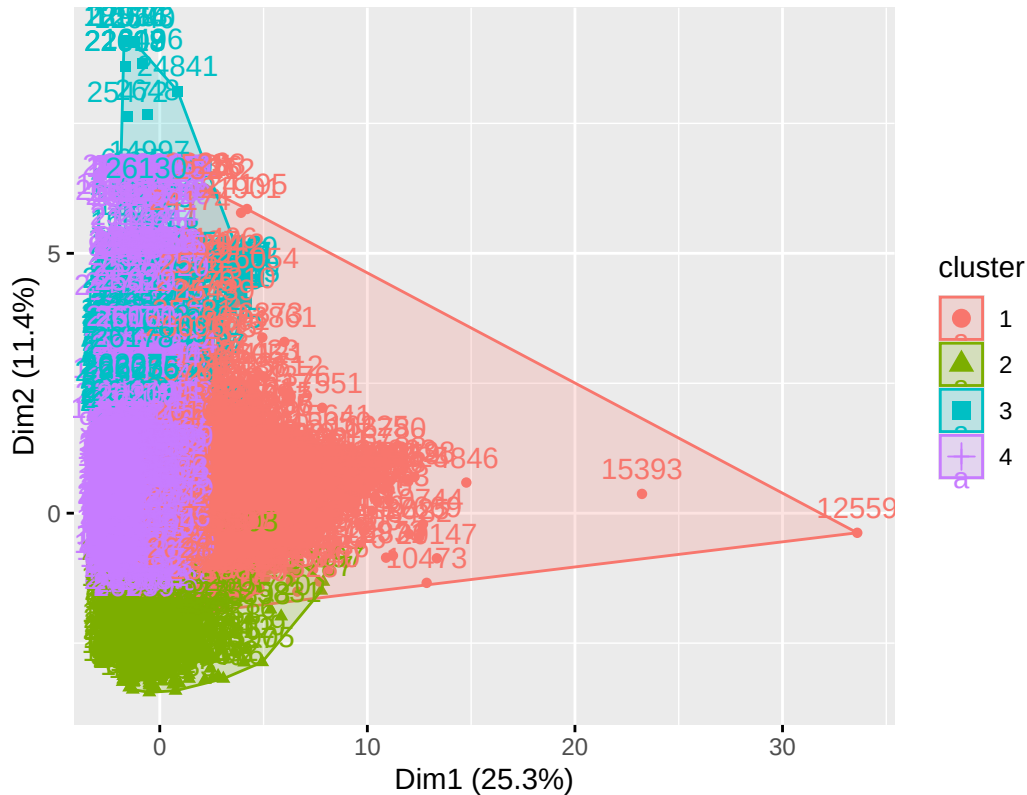
Elbow Method for Optimal K (Location Proxies)



```
# K-means Clustering (K = 4)
set.seed(123)
mktseg_kmeans_result_location <- kmeans(mktseg_cluster_data_scaled_location,
  centers = 4, nstart = 25)

fviz_cluster(mktseg_kmeans_result_location, data = mktseg_cluster_data_scaled_location,
  main = "K=4 Clustering Using Location Proxies")
```

K=4 Clustering Using Location Proxies



```
mktseg_clustered_location <- mktseg_cluster_data_location %>%
  mutate(segment = as.factor(mktseg_kmeans_result_location$cluster))
```

```
mktseg_LA_segmented_location <- mktseg_cluster_data_location %>%
  left_join(select(mktseg_clustered_location, row_id, segment), by = "row_id") %>%
  select(-row_id)
```

```
mktseg_segment_summary_location <- mktseg_LA_segmented_location %>%
  filter(!is.na(segment)) %>%
  group_by(segment) %>%
  summarise(avg_price = mean(price), avg_bedrooms = mean(bedrooms), avg_bathrooms = mean(bathrooms),
    avg_beds = mean(beds), avg_rating = mean(review_scores_rating,
    na.rm = TRUE), avg_availability = mean(availability_365), pct_downtown = mean(is_downtown),
    pct_westside = mean(is_westside), pct_eastside = mean(is_eastside),
    pct_valley = mean(is_valley))
```

```
print(mktseg_segment_summary_location)
```

```
## # A tibble: 4 x 11
##   segment avg_price avg_bedrooms avg_bathrooms avg_beds avg_rating
##   <fct>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 1          530.        3.55        2.82        4.52        4.82
## 2 2          208.        1.50        1.4         1.87        4.88
## 3 3          152.        1.13        1.58        1.67        4.69
## 4 4          146.        1.17        1.18        1.56        4.79
## # i 5 more variables: avg_availability <dbl>, pct_downtown <dbl>,
```

```
## #   pct_westside <dbl>, pct_eastside <dbl>, pct_valley <dbl>

silhouette_score_location <- silhouette(mktseg_kmeans_result_location$cluster,
    dist(mktseg_cluster_data_scaled_location))
plot(silhouette_score_location)
```

Silhouette plot of (x = mktseg_kmeans_result_loca

n = 26290

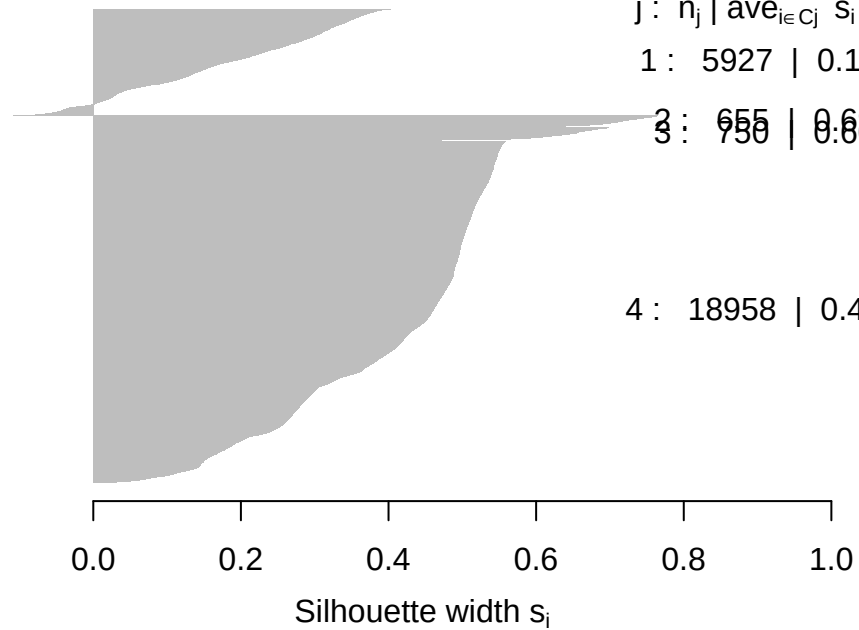
4 clusters C_j

j : n_j | $\text{ave}_{i \in C_j} s_i$

1 : 5927 | 0.18

2 : 655 | 0.69

4 : 18958 | 0.40



Average silhouette width : 0.36